

Threads and OpenMP

Finding Prime Numbers

Task 1 Serial Code (printPrimes) pt. 1

```
// Function to print prime numbers less than k
void printPrimes(int k, char *is_prime) {
    if (k <= 100) {
        // Print primes to console if k is less than or equal to 100
        printf("Primes less than %d: ", k);
        bool first = true;

        // Loop through the is_prime array and print the prime numbers
        for (int j = 2; j < k; j++) {
            if (is_prime[j] == 1) {
                if (!first) printf(", ");
                printf("%d", j);
                first = false;
            }
        }

        printf("\n");
    }
}
```

Print primes to console (standard output) if k is less than or equal to 100.

Task 1 Serial Code (printPrimes) pt. 2

```
} else if (k > 100) {  
    // Write primes to file if k is greater than 100  
    FILE *fptr;  
    char filename[] = "task1_output.txt";  
  
    fptr = fopen(filename, "w");  
  
    if (fptr == NULL) {  
        printf("Error: Could not create output file.\n");  
        return;  
    }  
  
    bool first = true;  
  
    // Loop through the is_prime array and write the prime numbers to the file  
    for (int i = 2; i < k; i++) {  
        if (is_prime[i] == 1) {  
            if (!first) {  
                fprintf(fptr, ", ");  
            }  
  
            fprintf(fptr, "%d", i);  
            first = false;  
        }  
    }  
  
    fprintf(fptr, "\n");  
    fclose(fptr);  
  
    printf("File '%s' written successfully.\n", filename);  
}  
}
```

Outputs primes to a text file (.txt) if k is more than 100.

Task 1 Serial Code (isPrime)

```
// Function to check if a number is prime
bool isPrime(int k) {
    if (k <= 1) {
        return false;
    } else if (k == 2) {
        return true;
    } else if (k % 2 == 0) {
        return false;
    }

    // Loop through odd numbers starting from 3 up to the square root of k
    // incrementing by 2 to skip even numbers
    for (int i = 3; i * i <= k; i += 2) {
        if (k % i == 0) {
            return false;
        }
    }

    return true;
}
```

Coded instant return values for 0, 1, 2, and even numbers.

Starts from 3 in the loop equivalently up to the square root of k.

Increments by 2 to skip even numbers.

Task 1 Serial Code (main) pt. 1

```
// Main function to execute the program
int main() {
    int k;
    struct timespec start, end;
    double time_taken;

    printf("Enter a number to find prime numbers smaller than it: ");
    scanf("%d", &k);

    if (k <= 2) {
        printf("There are no prime numbers less than %d.\n", k);
        return 1;
    }

    // Allocate memory for the is_prime array and initialize it to 0
    char *is_prime = (char *)calloc(k, sizeof(char));

    if (is_prime == NULL){
        printf("Memory allocation failed\n");
        return 1;
    }

    if (k > 2) {
        is_prime[2] = 1;
    }
}
```

Scans user input, returns early if the input is 2 or less.

Uses calloc to store primes.

Initialises the default value for is_prime array.

Task 1 Serial Code (main) pt. 2

```
// Get current clock time.
clock_gettime(CLOCK_MONOTONIC, &start);

// Loop through odd numbers starting from 3 up to k and check if they are prime
// incrementing by 2 to skip even numbers
for (int i = 3; i < k; i += 2) {
    if (isPrime(i)) {
        is_prime[i] = 1;
    }
}

// Get the clock current time again
// Subtract end from start to get the CPU time used.
clock_gettime(CLOCK_MONOTONIC, &end);
time_taken = (end.tv_sec - start.tv_sec) +
             (end.tv_nsec - start.tv_nsec) * 1e-9;
printf("Series prime search up to %d completed in: %lf seconds\n", k, time_taken);

printPrimes(k, is_prime);
free(is_prime);
}
```

Gets time before and after the computation to get the most accurate computational time.

Starts from 3 in the loop up to k.

Increments by 2 to skip even numbers.

Task 2 and 3 Basic info

Majority of the code is the same as task 1 for both task 2 and task 3.

Additional functions will be mentioned.

Prime storing and printing logic is the same.

Only difference is the computational logic so that measurements and testing will be more consistent (more constant variables as to not affect outcome).

Code is assumed to be the same if not mentioned.

Task 2 POSIX Threads (ThreadData struct)

```
// Structure to hold thread data
You, yesterday | 2 authors (loolishen and one other)
typedef struct {
    int thread_id;
    int k;
    char *is_prime;
} ThreadData;
```

Structure to hold thread data

Task 2 POSIX Threads (ThreadFunc)

```
// Thread function to check for prime numbers in a given range
void *ThreadFunc(void *pArg) {
    ThreadData *data = (ThreadData *)pArg;

    // This is to determine the starting point for each thread based on its thread ID
    int starting_point = 3 + (2 * data->thread_id);

    // This is to determine the step amount for each thread based on the number of threads
    int step_amount = 2 * NUM_THREADS;

    // Loop through the range of numbers assigned to this thread and checks if they are prime,
    // marking them in the is_prime array
    for (int i = starting_point; i < data->k; i += step_amount) {
        if (isPrime(i)) {
            data->is_prime[i] = 1;
        }
    }

    return NULL;
}
```

Thread function to check for prime numbers using the same logic as task 1.

Calculates a distinct starting point $3 + (2 * \text{thread_id})$ and stride $2 * \text{NUM_THREADS}$ to implement static partitioning.

Task 2 POSIX Threads (main)

```
// Fork
for (int i = 0; i < NUM_THREADS; i++) {
    thread_data[i].thread_id = i;
    thread_data[i].k = k;
    thread_data[i].is_prime = is_prime;

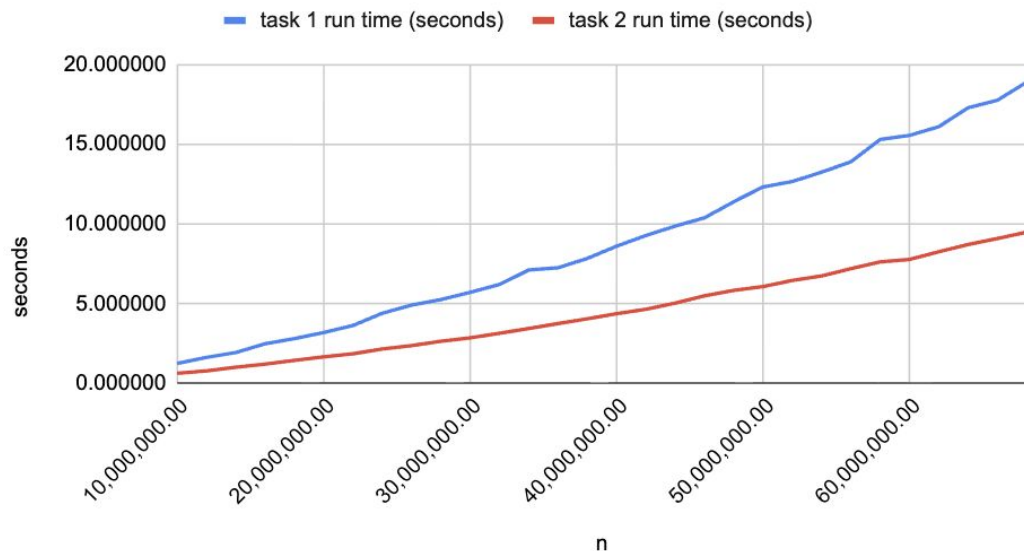
    pthread_create(&tid[i], NULL, ThreadFunc, &thread_data[i]);
}

// Join
for (int i = 0; i < NUM_THREADS; i++) {
    pthread_join(tid[i], NULL);
}
```

Basic loops to fork the process into multiple threads and joining back the results.

Task 2 POSIX Threads (graph 1)

Serial vs POSIX Threads run time with increasing n

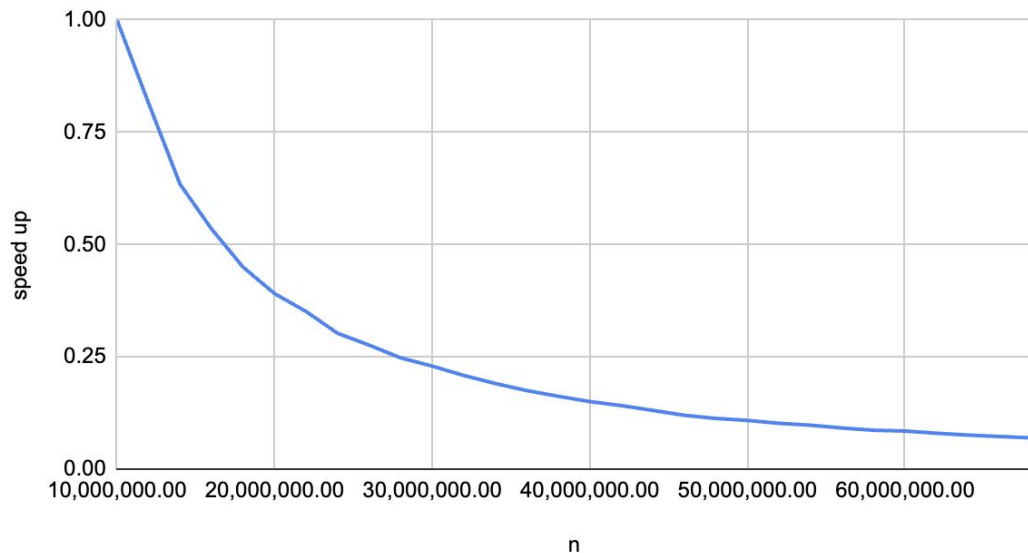


Task 2 was run with 2 threads for this graph comparison.

Task 1 is serial (runs on 1 thread).

Task 2 POSIX Threads (graph 2)

POSIX Threads speed up with increasing n



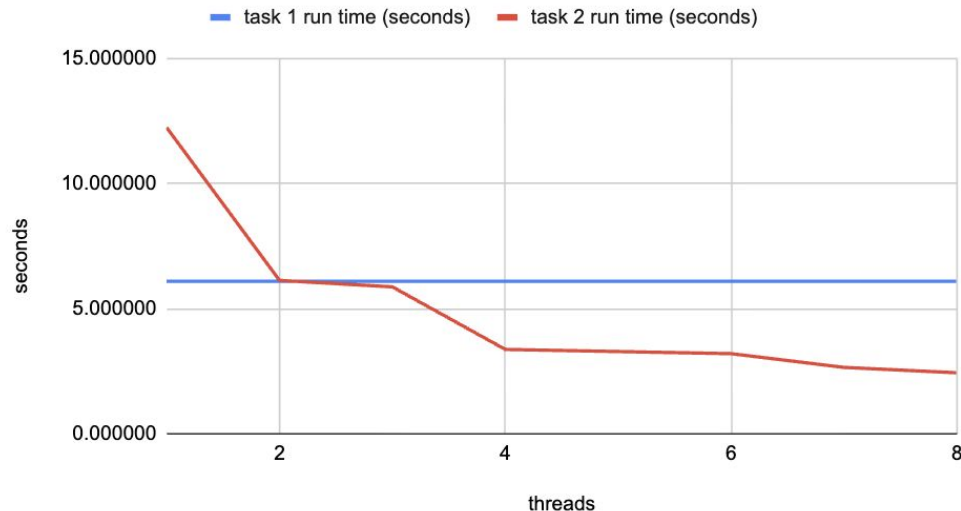
Task 2 was run with 2 threads for this graph comparison.

The speedup was calculated with the formula of base time at 10,000,000 n divided by the time at the current n.

$(\text{base time} / \text{current time})$

Task 2 POSIX Threads (graph 3)

Serial vs POSIX Threads run time with increasing threads



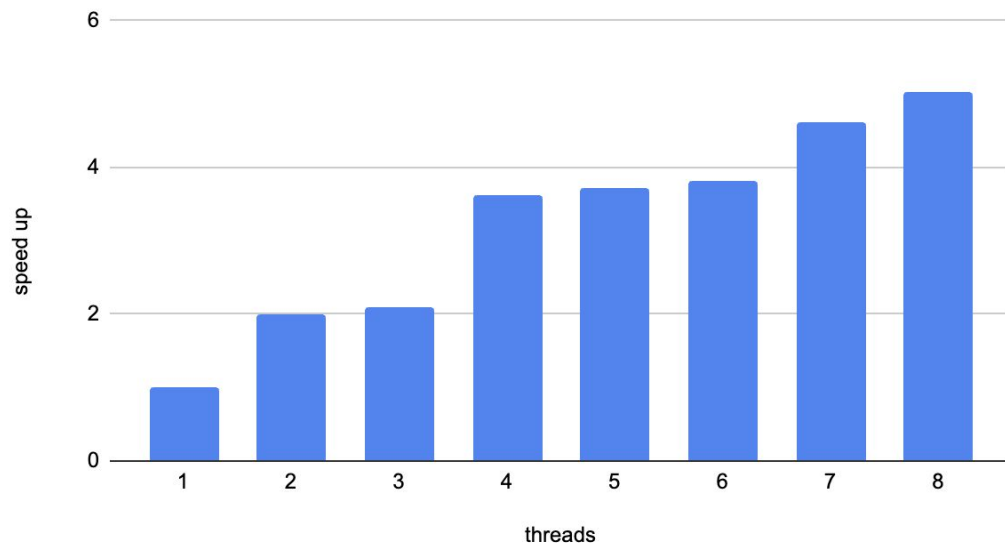
Task 1 is serial (runs on 1 thread).

Task 2 was run with increasing threads.

n was set at 50,000,000 for this graph comparison.

Task 2 POSIX Threads (graph 4)

POSIX Threads speed up with increasing threads



n was set at 50,000,000 for this graph comparison.

Task 3 OpenMP (main)

```
// Use OpenMP to parallelize the prime number search
#pragma omp parallel for schedule(guided)

// Loop through odd numbers starting from 3 up to k and check if they are prime
// incrementing by 2 to skip even numbers
for (int i = 3; i < k; i += 2) {
    if (isPrime(i)) {
        is_prime[i] = 1;
    }
}
```

Essentially the same as task 1 apart from the `pragma omp parallel` call to utilise OpenMP parallelization.

Guided partition scheme is used as prime finding has an uneven workload which guided is best at.

Task 3 OpenMP (graph 1)

Serial vs OpenMP run time with increasing n

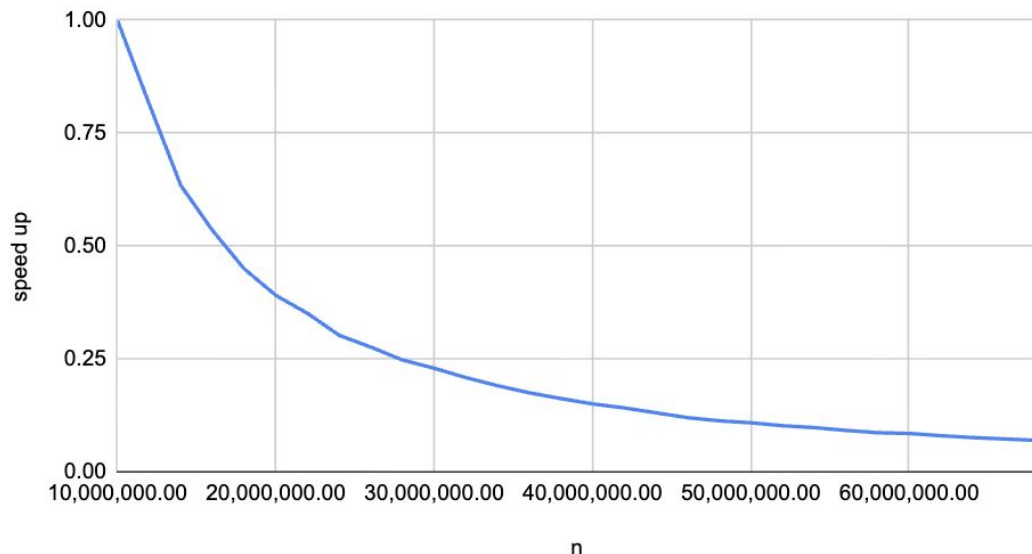


Task 3 was run with 2 threads for this graph comparison.

Task 1 is serial (runs on 1 thread).

Task 3 OpenMP (graph 2)

OpenMP speed up with increasing n



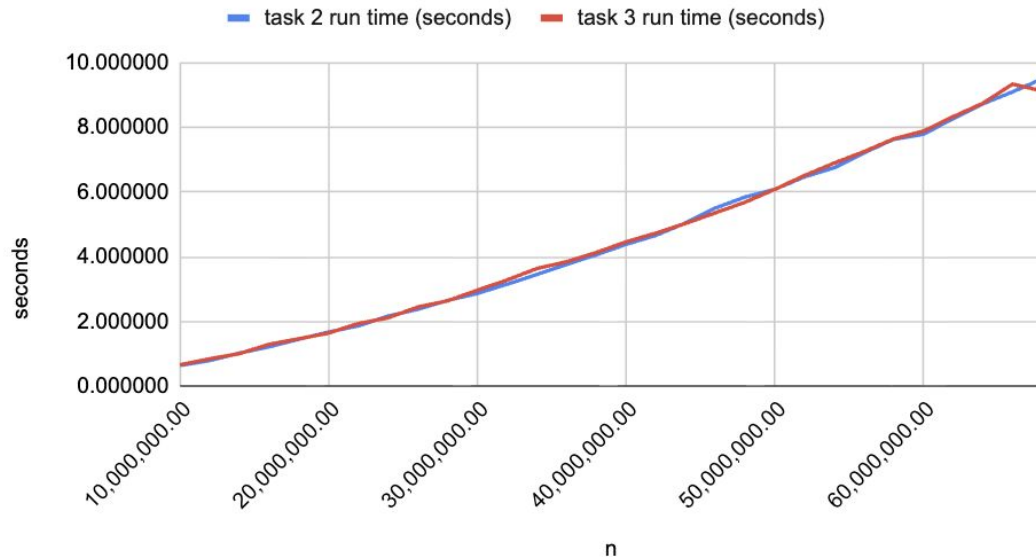
Task 3 was run with 2 threads for this graph comparison.

The speedup was calculated with the formula of base time at 10,000,000 n divided by the time at the current n.

$(\text{base time} / \text{current time})$

Task 3 OpenMP (graph 3)

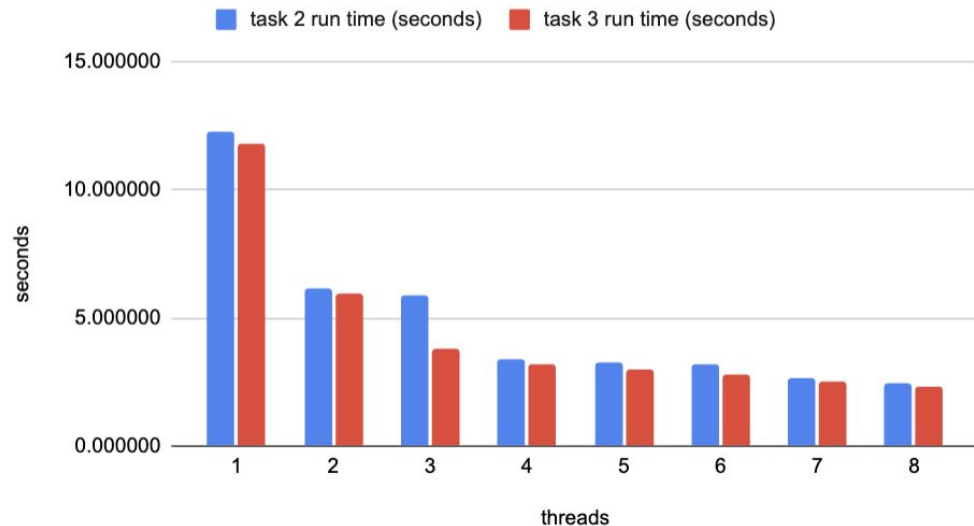
POSIX Threads vs OpenMP run time with increasing n



Both task 2 and task 3 was run with 2 threads for this graph comparison.

Task 3 OpenMP (graph 4)

POSIX Threads vs OpenMP run time with increasing n



n was set at 50,000,000 for this graph comparison.

Conclusion and Recommendations

Parallelization is extremely useful to fully utilise your machine's functions to speed up processes.

Use POSIX threads (pthreads) for parallelization if more control over parallelization is wanted.

Use OpenMP (omp) for parallelization if convenience is wanted as there are more inbuilt functions that are useful.

Do not use more threads than the cores available on your machine as the process will slow down instead (context switching is used in this scenario).

Q&A

Thank you for your attention